TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
KHWOPA COLLEGE OF ENGINEERING
Libali, Bhaktapur

[College logo – place image in `figures/`]

Department of Computer and Electronics Engineering

A REPORT ON

Project Title Goes Here

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF COMPUTER ENGINEERING

Submitted by:

Member One Name	KCE0XXBCT0XX
Member Two Name	KCE0XXBCT0XX
Member Three Name	KCE0XXBCT0XX
Member Four Name	KCE0XXBCT0XX

Under the Supervision of

Er. Supervisor Name
Lecturer
Department of Computer and Electronics Engineering
KHWOPA COLLEGE OF ENGINEERING

Libali, Bhaktapur, Nepal

Month, Year

CERTIFICATE OF APPROVAL

This is to certify that this minor project work entitled "Project Title Goes Here" submitted by Member One Name (KCE0XXBCT0XX), Member Two Name (KCE0XXBCT0XX), Member Three Name (KCE0XXBCT0XX) and Member Four Name (KCE0XXBCT0XX) has been examined and accepted as the partial fulfillment of the requirements for the degree of BACHELOR OF COMPUTER ENGINEERING. The project was carried out under special supervision and within the time frame prescribed by the syllabus.

Er. External Examiner Name
External Examiner
Senior Lecturer
Department of Computer and Electronics Engineering
External College Name

Er. Supervisor Name
Project Supervisor
Lecturer
Department of Computer and Electronics Engineering
KHWOPA COLLEGE OF ENGINEERING

Er. HOD Name
Head of Department
Department of Computer and Electronics Engineering
KHWOPA COLLEGE OF ENGINEERING

COPYRIGHT©

ACKNOWLEDGEMENT

Member One Name	KCE0XXBCT0XX
Member Two Name	KCE0XXBCT0XX
Member Three Name	KCE0XXBCT0XX
Member Four Name	KCE0XXBCT0XX

ABSTRACT

Keywords:

Contents

List of Figures

List of Tables

LIST OF SYMBOLS AND ABBREVIATION

API	Application Programming Interface
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
GPU	Graphics Processing Unit
RAM	Random Access Memory
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Background

Artificial intelligence and video games have been studied together for decades. In the early years of computing, games were treated as ideal testbeds for autonomous decision-making. Shannon [1] pointed to chess as an optimal research environment because of its well-defined state space, explicit rules and unambiguous terminal goal, and Samuel [2] soon after showed that an automated system could surpass human expert play in checkers through adaptive evaluation functions. By the 1980s and 1990s, game AI was common in commercial titles, but it depended largely on static, deterministic techniques such as Finite State Machines (FSMs) and scripted decision trees. These were lightweight and reliable in predictable settings, yet they generalized poorly: an unexpected player strategy or a small change in the environment was often enough to make the whole policy fail.

The situation changed with modern machine learning, and particularly with Deep Reinforcement Learning (DRL), where an agent learns a control policy by trial and error through interaction with its environment. A key breakthrough came from Mnih *et al.* [3], who combined deep convolutional networks with Q-learning in the Deep Q-Network (DQN). This architecture linked high-dimensional visual input directly to action selection, producing end-to-end control policies learned from raw pixel arrays across the Atari 2600 benchmark suite.

Despite these successes, DRL carries real operational costs. Deep models need large amounts of experience, considerable hardware and long training runs, especially when they operate on high-dimensional raw pixel matrices. They also depend on a fixed, hand-engineered network topology that is decided before training begins. Since the structure itself cannot adapt, such models struggle to scale to dynamic control domains. In a fast-paced platforming game like Super Mario Bros., where continuous momentum, split-second collision avoidance and spatial pathfinding all have to be resolved at once, fixed-topology DRL models run into both convergence and computational bottlenecks.

Neuroevolution offers a different route. Instead of tuning connection weights over a fixed architecture through gradient descent, it evolves network topologies and parameters together using evolutionary algorithms. The foundational method in this area is NeuroEvolution of Augmenting Topologies (NEAT), introduced by Stanley and Miikkulainen [4]. NEAT begins with a population of minimal networks and complexifies

them incrementally, which keeps the early generations searching in a low-dimensional parameter space before any structural capacity is added.
This project applies NEAT to Super Mario Bros. to examine whether dynamic structural self-organization allows an agent to evolve competent platforming behaviour and generalize across level variations, without hand-crafted heuristics and without the overhead of a pixel-based DRL pipeline.

1.2 Motivation

The motivation behind this project is to build lightweight, adaptive control agents with NEAT as an alternative to compute-heavy Deep Reinforcement Learning systems. Rather than depending on static deep architectures that need substantial GPU acceleration, NEAT evolves weights and network topology at the same time, starting from a minimal baseline and adding structure only when the task actually demands it.
Within that framework, the work focuses on making NEAT efficient in a platforming environment through two targeted enhancements: screen downsampling and automated speciation. Both serve a single aim, which is a lightweight NEAT-based agent for Super Mario Bros. that relies on direct RAM state extraction and spatial quantization to achieve real-time control on ordinary consumer hardware.

1.3 Problem Statement

Automating a platforming game such as Super Mario Bros. for high-speed completion demands continuous momentum control, precise collision avoidance and sound pathfinding under tight timing constraints, and applying neuroevolution here runs into three connected difficulties. Feeding raw frame buffer output into the network, a 256×240 pixel matrix amounting to 61,440 input features per frame, inflates the input layer and with it the parameter search space, which slows population evaluation to the point where structural evolution becomes impractical. The reward landscape is also deceptive: standard fitness measures often reward agents that simply stay put in a safe spot and avoid damage, penalizing the multi-step risks such as leaping a bottomless pit or slipping past an enemy that progress actually requires, so the search settles into local optima. Finally, structural mutations tend to be selected away before they can prove useful, because adding a node or connection disturbs weights that were already working and lowers fitness in the short term; without a mechanism that shelters new topologies while they are being optimized, evolution consistently favours established, fully tuned networks over genuine innovation.

1.4 Objective

- To develop a lightweight NEAT-based agent for Super Mario Bros. that uses direct RAM state extraction and spatial quantization to achieve efficient real-time control on standard consumer hardware.

1.5 Scope and Applications

1.5.1 Scope

The project covers the design, training and evaluation of a NEAT agent on the early stages of Super Mario Bros., mainly Level 1-1, running on the stable retro NES emulator. The agent reads its state from emulator RAM instead of from pixels, and NEAT is the only control algorithm implemented; DQN and PPO appear in this report for comparison but are not built. Training is containerized with Docker for cross-platform reproducibility and stays CPU-bound throughout, so no GPU is required at any stage.

1.5.2 Applications

The same approach transfers to other settings where a control policy has to be learned cheaply. It suits game developers who want non-scripted opponents or playtesting bots that can find unintended routes and exploits before release, and it gives researchers and students a neuroevolution testbed that runs on a laptop rather than a cluster. More broadly, the combination of a compact state representation with a topology that grows only as needed is useful for embedded and resource-limited control problems, such as simple robotic navigation, where memory and processing budgets rule out large fixed networks.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Review

2.1.1 Classical and Heuristic Game AI

In the early development of video game artificial intelligence, control policies rested almost entirely on deterministic, human-designed frameworks such as Finite State Machines (FSMs), behaviour trees and heuristic search. The groundwork for treating games as serious AI problems was laid much earlier: Shannon [1] framed chess as an ideal research domain because of its well-defined state space and unambiguous goal, and proposed the minimax evaluation that underpins most later search agents. Samuel [2] went a step further with checkers, showing that a program could improve its own evaluation function through self-play and rote learning, and eventually outplay its author. Both results established the pattern that games would remain the benchmark of choice for machine decision-making.

In platforming games specifically, tree search has produced some of the strongest classical results. The clearest example came from the 2009 Mario AI Competition described by Togelius, Karakovskiy and Baumgarten [12], where Baumgarten's A* agent took first place by planning through an internal forward model of the game. The agent searched over future game states in the space of jump and movement actions, and because the simulator it planned with was an exact copy of the game engine, its plans were near-optimal. It cleared levels faster and more reliably than every learning-based entry in that year's competition.

That success, however, depends on conditions that rarely hold outside a competition harness. Two limitations stand out:

1. **Model Dependence:** A* and related planners need real-time access to a perfect internal simulator in order to predict future frames. Where the dynamics are unknown or parts of the execution state are hidden, the planner has nothing to search over and the method fails outright.
2. **Brittle Generalization:** Hand-crafted decision trees and tuned heuristic weights do not adapt when environmental parameters change. A shift in level geometry or in enemy spawn patterns during execution is often enough to break behaviour that was reliable a moment earlier.

2.1.2 Deep Reinforcement Learning Paradigms

Combining deep neural networks with reinforcement learning shifted the field towards policies learned directly from sensory observation rather than written by hand. Mnih *et al.* [3] introduced the Deep Q-Network, which paired a convolutional network with Q-learning and learned to play forty-nine Atari 2600 games from raw pixels and the score alone, reaching or exceeding professional human level on a majority of them using one architecture and one set of hyperparameters throughout. Two mechanisms made this stable: experience replay, which stores transitions and samples them at random to break the correlation between consecutive frames, and a periodically frozen target network that keeps the regression target from moving with every update.

Later policy-gradient methods improved on this. Schulman *et al.* [7] proposed Proximal Policy Optimization, which constrains each policy update with a clipped surrogate objective so that a single batch of experience can be reused for several gradient steps without the update collapsing the policy. PPO gave much of the reliability of earlier trust-region methods at a fraction of the implementation cost, and it became a default choice for continuous control.

Deceptive and sparse rewards remained a separate problem. Pathak *et al.* [8] addressed it with curiosity as an intrinsic reward, defining novelty as the error of a forward model predicting the consequences of the agent's own actions in a learned feature space that deliberately ignores parts of the observation the agent cannot influence. Their agent learned to explore Super Mario Bros. with no external reward at all, clearing a substantial portion of the first level purely to reach screen states it could not yet predict.

Applied to platformers, though, the paradigm carries structural costs:

- **High Sample Complexity:** Training routinely needs tens of millions of environment interactions before basic momentum and collision behaviour becomes reliable.
- **Fixed Topology Constraints:** The network architecture is chosen before training and never changes. Optimization adjusts weights only, so structural capacity stays locked regardless of how hard the task turns out to be.
- **Heavy Computational Requirements:** Pushing uncompressed 256×240 frame buffers through multi-layer convolutional networks calls for dedicated GPU acceleration and a substantial energy budget.

2.1.3 Neuroevolution and NEAT Milestones

Neuroevolution takes a different view of the same problem, treating policy generation as a global stochastic search rather than a gradient descent. Yao [9] surveyed the field as it stood before NEAT and drew the distinction that still organizes it: evolving connection weights, evolving architecture, and evolving learning rules are separate levels at which evolution can act, and combining them is what makes evolutionary methods more than an alternative optimizer. Yao also identified the permutation problem, where two networks encode the same function through different node orderings, which makes naive crossover between architectures destructive.

Stanley and Miikkulainen [4] answered that problem directly with NeuroEvolution of Augmenting Topologies. NEAT co-evolves connection weights and graph topology from a minimal starting configuration, and rests on three ideas that work together. Historical markings assign every new gene a global innovation number, so two genomes can be aligned by origin and crossed over without the permutation problem. Speciation groups genomes by topological similarity and makes them compete mainly within their own species, which gives a newly added node or connection time to be optimized before it has to justify itself against fully tuned rivals. Starting minimal and complexifying incrementally keeps the search in a low-dimensional space early on and adds dimensions only when they earn their place. Their ablation experiments showed each of the three is necessary; removing any one of them substantially degrades performance.

Lehman and Stanley [5] later questioned the objective itself. They argued that in deceptive domains a fitness function actively misleads search, because the stepping stones to a solution usually look worse than the local optimum next to them. Their alternative, novelty search, rewards behavioural novelty and ignores the objective entirely, and it outperformed objective-based search on deceptive maze and biped tasks. The observation applies directly to platforming, where fitness measured as horizontal progress penalizes exactly the risky multi-step manoeuvres that progress requires.

NEAT reached platforming games most visibly through MarI/O [10], an open-source demonstration that ran NEAT through Lua scripting in the BizHawk emulator and evolved a network that completed a level of Super Mario World from a tile-grid input. It showed convincingly that minimal evolved topologies could solve a control task of this kind, but the implementation had clear constraints:

1. **Single-Level Overfitting:** Networks were evolved against one fixed level, so agents memorized specific button timings for that layout instead of acquiring platforming behaviour that transferred elsewhere.

2. **Brittle Fitness Metrics:** The reward was dominated by simple horizontal progress, which left agents prone to local optima such as standing still in a safe pocket of the level or stalling against a minor obstacle.

Autin [11] built a more careful NEAT platforming framework in *Super Mario Evolution by the Augmentation of Topology* to address these weaknesses. The fitness evaluation was refined to penalize time stagnation and risk aversion rather than rewarding raw distance, and training moved to a scenario-based regime in which evolving networks face a range of environmental setups instead of a single fixed level. Together these changes pushed the evolved agents towards generalized obstacle traversal rather than level-specific memorization.

2.1.4 State Representation and Direct RAM Extraction

How the input state is represented governs both agent performance and, just as importantly, evaluation throughput. The Mario AI Benchmark of Karakovskiy and Togelius [6] made the point concretely: the framework was built so that agents could be evaluated at many times real-time speed across procedurally generated levels, and feeding uncompressed visual frame buffers into evolving networks inflates the input layer to the point where parameter counts explode and real-time population evaluation stops being tractable. Their competition results also showed that agents tested only on the levels they were tuned for scored well above their performance on unseen ones, which is the same overfitting failure that later affected MarI/O.
Modern implementations therefore tend to bypass frame rendering in favour of reading the emulator's memory directly:

- **Direct RAM Extraction:** Memory-mapping utilities such as those developed by Yu [13] for Super Mario Bros. reinforcement learning environments read entity byte registers straight from emulator RAM. Player coordinates (x, y), velocity vectors and active sprite hitboxes come out as exact values with no rendering or computer vision overhead, and without the ambiguity of inferring them from pixels.
- **Spatial Memory Quantization:** Mapping continuous RAM coordinates onto a discrete, agent-centric grid, such as a 16×13 tile matrix, compresses a high-dimensional spatial observation into a small set of structural categories, typically solid terrain as $+1$, hostile entities as -1 and empty space as 0. The result is a low-latency input small enough for a minimal NEAT topology to work with from the first generation.

2.1.5 Identified Research Gap

Taken together, the literature leaves a specific opening. Classical planners are efficient but need a perfect forward model and do not generalize. Deep reinforcement learning generalizes better but pays for it in sample complexity and GPU time, and its topology is fixed before the task is understood. NEAT removes the fixed-topology constraint, yet the best-known platforming applications of it either overfit a single level, as MarI/O did, or address fitness design without also addressing the cost of the input representation. The combination this project pursues, direct RAM extraction with spatial quantization feeding a speciated NEAT population trained entirely on CPU, sits in that gap: it targets the input dimensionality problem identified by Karakovskiy and Togelius [6], the deceptive fitness problem raised by Lehman and Stanley [5], and the premature elimination of structural innovation that Stanley and Miikkulainen's speciation mechanism [4] was designed to prevent.

2.2 Comparative Analysis of Control Paradigms

Table 2.1 summarizes the architectural and operational trade-offs across classical AI, deep reinforcement learning, early neuroevolution as represented by MarI/O, and the advanced topology evolution adopted in this project.

Table 2.1: Comparison of Autonomous Game-Playing Approaches

Dimension	**Classical AI**	**Deep RL**	**Early NEAT**	**Advanced NEAT**
Architecture	Static rules	Fixed neural layers	Dynamic topology	Dynamic topology
Compute hardware	Low (single CPU)	High (dedicated GPU)	Low (single CPU)	Moderate (multi-core CPU)
Input representation	Direct state engine	Uncompressed pixels	Basic screen matrix	Quantized 16×13 RAM grid
State extraction	Forward simulator	Visual frame buffer	Emulator Lua hook	Direct memory scraper
Generalization ability	Poor (brittle rules)	Moderate (sample hungry)	Poor (level memorization)	High (fitness and dataset refined)
Training efficiency	Instant	10^7+ frames	10^5+ evaluations	10^4–10^5 evaluations

2.3 Review Matrix

Table 2.2: Review Matrix of the Surveyed Literature

Reference	Problem Statement	Methodology	Outcomes	Research Gaps
1. Shannon [1] (1950), Programming a Computer for Playing Chess	Whether a machine can play chess without exhaustively enumerating a search space too large to traverse	Minimax search to a fixed depth with a hand-designed evaluation function over material and position	Established the evaluation-plus-search formulation used by later game-playing programs	Evaluation weights set by hand; no mechanism for the program to improve them from experience
2. Samuel [2] (1959), Machine Learning Using the Game of Checkers	Hand-tuned evaluation functions do not improve and cannot exceed the knowledge of their designer	Self-play with rote learning and adjustment of evaluation-function weights from game outcomes	Program improved beyond its author's play; first widely cited demonstration of machine learning in a game	Restricted to a small discrete domain with a linear evaluation function; no learning from raw perception
3. Togelius, Karakovskiy & Baumgarten [12] (2010), The 2009 Mario AI Competition	Whether platforming can be automated well by planning, and how competing approaches compare on a common benchmark	Open competition on a Super Mario clone; the winning A* agent planned over an internal forward model of the engine	A* agent won first place and outperformed all learning-based entries on level completion and speed	Requires an exact simulator to plan with; agents were brittle to unseen level geometry and enemy patterns

Continued on next page

Table 2.2 continued from previous page

Reference	Problem Statement	Methodology	Outcomes	Research Gaps
4. Mnih *et al.* [3] (2015), Human-Level Control through Deep Reinforcement Learning	Reinforcement learning from high-dimensional raw sensory input was unstable and domain-specific	Deep Q-Network combining a convolutional network with Q-learning, experience replay and a periodically frozen target network	Human-level or better play on the majority of 49 Atari 2600 games from pixels alone, with one architecture throughout	Tens of millions of frames per game; fixed topology chosen before training; GPU dependent
5. Schulman *et al.* [7] (2017), Proximal Policy Optimization Algorithms	Policy-gradient updates are unstable, and trust-region methods that fix this are complex and costly to implement	Clipped surrogate objective limiting policy change per update, allowing several epochs of minibatch reuse per batch	Stability comparable to trust-region methods at far lower complexity; strong results on continuous control and Atari	Still fixed topology and sample-hungry; no structural adaptation to task difficulty
6. Pathak *et al.* [8] (2017), Curiosity-Driven Exploration by Self-Supervised Prediction	Sparse and deceptive external rewards leave agents with no gradient to explore along	Intrinsic reward from forward-model prediction error in a learned feature space that ignores uncontrollable observation detail	Agent explored Super Mario Bros. and cleared much of level 1-1 with no external reward; improved exploration in VizDoom	Curiosity can stall on stochastic distractors; still built on a fixed deep architecture with heavy compute demands

Continued on next page

Table 2.2 continued from previous page

Reference	Problem Statement	Methodology	Outcomes	Research Gaps
7. Yao [9] (1999), Evolving Artificial Neural Networks	Network architecture is normally fixed by hand, and evolving it naively is disrupted by competing conventions	Survey and taxonomy separating the evolution of weights, architectures and learning rules, with their interactions	Framed evolution at multiple levels as a coherent alternative to gradient training; named the permutation problem	No concrete encoding solving crossover between differing topologies; largely pre-empirical for control tasks
8. Stanley & Miikkulainen [4] (2002), Evolving Neural Networks through Augmenting Topologies	Evolving topology and weights together suffers from destructive crossover and early loss of new structure	NEAT: historical markings for gene alignment, speciation to protect innovation, and incremental complexification from a minimal start	Outperformed fixed-topology neuroevolution on pole balancing; ablation showed all three mechanisms are necessary	Demonstrated on low-dimensional control; input representation for high-dimensional game states left open
9. Lehman & Stanley [5] (2011), Abandoning Objectives	In deceptive domains the fitness function itself misleads search away from the stepping stones to a solution	Novelty search rewarding behavioural novelty against an archive, with the objective removed from selection entirely	Outperformed objective-based search on deceptive maze navigation and biped walking tasks	Pure novelty scales poorly as behaviour space grows; combining novelty with an objective left to later work

Continued on next page

Table 2.2 continued from previous page

Reference	Problem Statement	Methodology	Outcomes	Research Gaps
10. SethBling [10] (2015), MarI/O	Whether NEAT can evolve a working platforming controller in a real commercial game	NEAT via Lua in the BizHawk emulator, with a tile-grid input read from the emulator and fitness based on horizontal progress	Evolved a network that completed a level of Super Mario World from a minimal starting topology	Single-level overfitting and memorized button timings; progress-only fitness traps agents in local optima
11. Karakovskiy & Togelius [6] (2012), The Mario AI Benchmark and Competitions	Platforming agents lacked a common benchmark, and were being evaluated on the levels they were tuned for	Benchmark on procedurally generated levels supporting faster-than-real-time evaluation, across gameplay and learning tracks	Standardized comparison across competition entries; quantified the gap between seen and unseen level performance	Raw frame input inflates the input layer and makes population evaluation intractable; generalization remained weak
12. Yu [13] (2020), Super Mario Bros. RL with RAM State Extraction	Pixel-based state extraction adds rendering and vision overhead and yields imprecise entity positions	Direct mapping of emulator RAM addresses to player coordinates, velocities and sprite hitboxes for RL environments	Exact low-dimensional state obtained without rendering, cutting per-step cost substantially	Address maps are game-specific and hand-derived; not paired with an evolving topology or a speciated population

Continued on next page

Table 2.2 continued from previous page

Reference	**Problem Statement**	**Methodology**	**Outcomes**	**Research Gaps**
13. Autin [11] (2024), Super Mario Evolution by the Augmentation of Topology	Early NEAT platforming agents memorized single levels and were misled by progress-based fitness	Refined fitness penalizing time stagnation and risk aversion, with scenario-based training across varied environmental setups	Agents generalized to obstacle traversal beyond a single level, improving on MarI/O-style implementations	Input representation and CPU-bound throughput not treated jointly with the fitness redesign

CHAPTER 3

METHODOLOGY

3.1 System Block Diagram

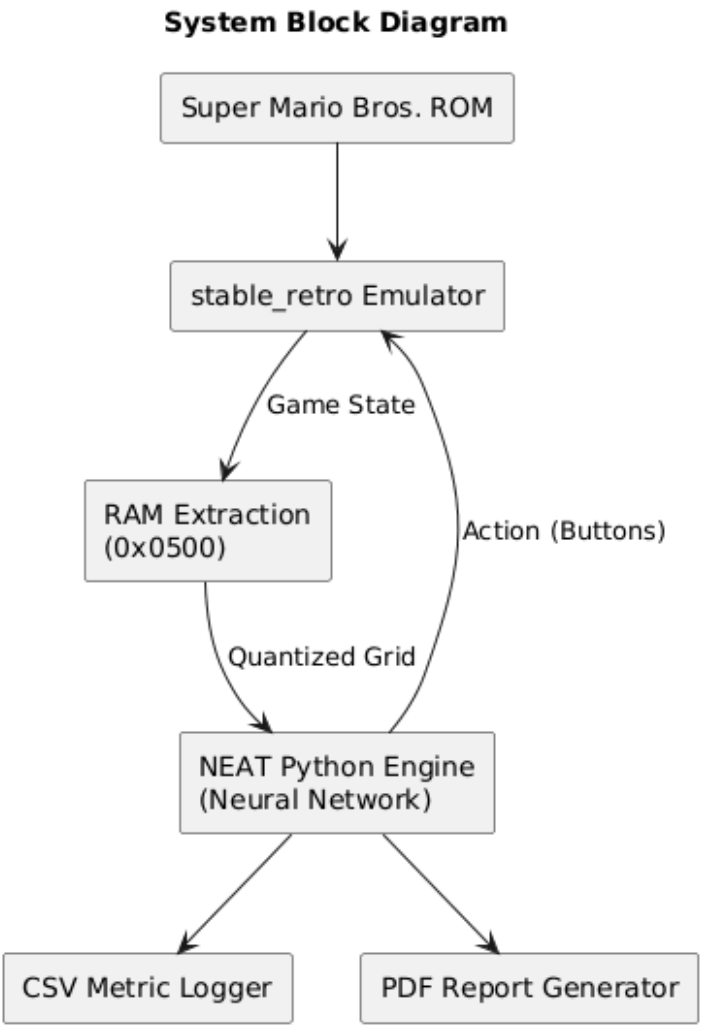


Figure 3.1: System Block Diagram

Figure 3.1 shows how the parts of the system fit together. The game ROM is loaded into the emulator, the emulator's memory is read every frame and reduced to a small grid, that grid is fed to the evolving network, and the network's decision is sent back to the emulator as a button press. This loop repeats for the whole run. Everything to the side of the loop, the logger and the report generator, only observes; nothing there feeds back into the agent.

3.2 Overall Approach

We treated this project as an experiment rather than as a single implementation. The question we set out to answer was whether NEAT can evolve an agent that plays Super Mario Bros. competently, and we found early on that the answer depends heavily on what the agent is asked to do during training. An agent evolved against one level becomes very good at that level and close to useless anywhere else. That result is easy to obtain and easy to misread as success, so we decided to treat it as a baseline rather than as an outcome.

We therefore built one shared pipeline and ran three different training strategies through it. The pipeline handles everything that does not depend on the strategy: loading the ROM, reading the emulator's memory, turning that memory into network inputs, converting network outputs into button presses, and recording what happened. The three strategies differ only in what a genome is evaluated on and how its fitness is computed from those evaluations.

The first strategy trains a separate population per level and gives us a specialist for each. The second evaluates one genome across several levels and combines the results into a single fitness, which asks for consistency instead of peak performance. The third departs further from both: instead of starting every evaluation at the opening frame of a level, we collected a library of mid-level situations and evaluated genomes from those, so that the difficult parts of a level are practised as often as the easy opening stretch. This last approach follows the scheme proposed by Autin (2024) and is the one we developed furthest.

Keeping the infrastructure common across all three matters for the comparison. Because the state representation, the action decoding and the logging are identical in every run, any difference in the results comes from the training strategy itself and not from an incidental difference in how the agent sees the game.

3.3 Common System Architecture

This section describes the parts of the system that every approach uses unchanged.

3.3.1 Game ROM and Emulator

We used the original Super Mario Bros. NES ROM running under `stable_retro`, a maintained fork of OpenAI Retro. We chose an emulator over reimplementing the game because the emulator reproduces the original physics exactly. Momentum in this game is subtle, and an approximation would have changed the problem the agent has to solve without us being able to say by how much.
Each genome is evaluated in its own emulator instance, created with `stable_retro.make()` and closed when the episode ends. Keeping the instances separate means no state can leak from one genome's run into the next. We train headless, with rendering disabled, because nothing in the pipeline reads the video output and drawing frames would only cost time. Episodes are capped at 4500 frames, roughly 75 seconds of game time, which is long enough for a capable genome to finish a level and short enough that a hopeless one does not tie up a core.

3.3.2 Game State Extraction from RAM

Rather than reading the screen, we read the emulator's memory directly. This was a deliberate methodological choice. Extracting state from pixels would mean adding a convolutional front end whose only job is to recover information the game already stores explicitly, and that front end would dominate both the parameter count and the training cost. Reading memory gives us the same information exactly, with no inference step and no rendering overhead.
Every frame we take a snapshot of the 2 KB of NES work RAM and read the values that describe the current situation:

- **Mario's position.** His position in the level is assembled from a page byte at `0x6D` and an offset at `0x86`. His on-screen pixel coordinates come from `0x03AD` and `0x03B8`.
- **Velocity.** Horizontal speed is a signed byte at `0x7D`. This one is read for scoring only and is never given to the network: it supplies a fitness term and the guard against a degenerate behaviour described in Section 3.4.2.
- **Enemies.** The game tracks five enemy slots. An activity flag for each starts at `0x0F`, with coordinates starting at `0x6E` and `0xCF`.
- **Death and completion.** Remaining lives sit at `0x075A`, the player state byte at `0x000E` signals the dying animation, and a viewport byte at `0x00B5` reveals a

fall into a pit. The byte at `0x001D` reads 3 when Mario is on the flagpole, which is how we detect a completed level.

- **Tilemap.** The background occupies a block beginning at `0x0500`, stored as two 16-wide pages covering 32 columns by 13 rows.

Because a tile's address is simple arithmetic on its coordinates, we compute the whole address table once when the module loads and reuse it for the rest of the run. Each frame's read is then a single vectorised lookup rather than several hundred individual ones, which matters when the read happens sixty times a second for every genome in the population.

3.3.3 Spatial State Representation

The extracted values are assembled into a grid that becomes the network's input. The reduction is as follows:

$$256 \times 240 \text{ pixel frame}$$
$$\downarrow$$
$$\text{RAM tile representation}$$
$$\downarrow$$
$$16 \times 13 \text{ tile grid}$$
$$\downarrow$$
$$208 \text{ input features}$$

The grid covers what is currently on screen and is flattened into a 208-element vector, which is the network's entire input. No scalar values are appended to it: the grid already encodes Mario's position, the terrain and the enemies, which is everything needed to choose a button. Each cell carries one of four values:

0	empty space
1	solid terrain
-1	enemy
2	Mario

This is the step that makes the project runnable on ordinary hardware. A raw frame carries 61,440 values; the grid carries 208, a reduction of roughly 295 times. What it discards is colour, sprite artwork and animation detail, none of which affects whether a jump clears a gap. What it keeps is where the ground is, where the enemies are and where Mario stands, which is what the decision actually depends on.

One detail here was a correction rather than a design decision. Mario is written into the grid at his own tile position, but during a jump his pixel height rises above the top row and the computed row index goes negative. Our original bounds check silently

dropped him from the grid whenever that happened, so the network lost sight of Mario at the apex of every jump, which is precisely when that information matters most. We now clamp the row to the nearest edge instead, which keeps him present for the whole arc while leaving his horizontal position exact.

3.3.4 NEAT-Based Agent and Action Generation

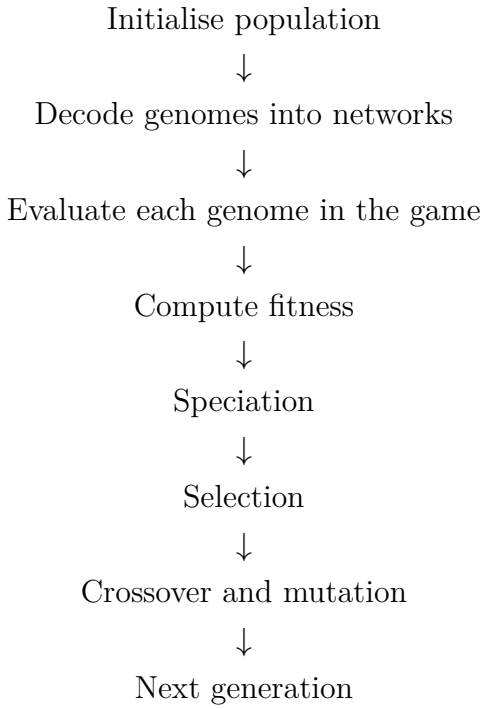
The agent is not a fixed model. NEAT is the evolutionary algorithm underneath it, and each genome in the population encodes a neural network whose topology as well as its weights are subject to evolution. What we specify is the interface: 208 inputs and 6 outputs. Everything between them is discovered during training, so it is more accurate to speak of an evolutionary neural agent than of an architecture we designed.
Each of the six outputs corresponds to one controller button: RIGHT, LEFT, A, B, UP and DOWN. A button counts as held whenever its output exceeds 0.5. Because the outputs are thresholded independently, several can be active at once, which is what lets the agent express combinations such as running right while jumping. Most obstacles in the game cannot be cleared without one.
We resolve button names against the environment's own button list at runtime rather than hard-coding positions in the action array, since that ordering is a property of the emulator rather than of the agent.

3.3.5 Fitness Function

Every approach scores its genomes with a fitness function, since fitness is what decides which genomes reproduce. The function itself is not common to the three approaches, however: each one asks a different question of its genomes and therefore needs a different way of measuring them, and the fitness function is the main thing that distinguishes them. The three formulations are given alongside the approaches that use them in Section 3.4.

3.3.6 Evolutionary Process

One generation proceeds as follows:



We run a population of 150. Genomes start fully connected, every input wired to every output, with no hidden nodes; structure is added later by mutation. Each genome is decoded into a feed-forward network, played through an episode and scored. Genomes are then grouped into species by structural similarity so that a new topology competes against close relatives rather than against the whole population before its weights have been tuned. The top 20% of each species breeds, the best two genomes carry over unchanged, and the children form the next generation.
Because each genome's episode is independent of every other, we evaluate them in parallel across CPU cores. We wrote our own parallel evaluator rather than using the one in `neat-python`, whose worker contract expects a worker to return a fitness for a genome handed to it. Our workers each have to build an emulator instance, run the episode and tear the emulator down again, which does not fit that contract.

3.3.7 Logging, Checkpoints and Evaluation

Recording runs properly mattered more than we expected, because training takes hours and a run whose history is lost has to be repeated. Two consumers sit at the end of the pipeline and observe without influencing anything.
A CSV logger is attached to the population as a reporter and fires at the end of every generation, appending one row to `metrics.csv` containing:

- generation number, timestamp and duration;
- fitness statistics, namely maximum, mean, median, minimum and standard deviation;

- game statistics, namely the best distance reached and whether the level was completed;
- topology statistics for the best genome, its node and connection counts, together with the number of species alive.

Checkpoints are written every five generations. Alongside them we save the champion genome separately, because NEAT's own checkpoint stores the next generation's unevaluated children and would otherwise discard the evaluated best genome of the generation that just finished.
A PDF reporter fires on the same interval and produces a visual summary using `matplotlib`, which works headless and therefore inside Docker. Each report covers fitness and topology at a high level, a diagram of the champion network, that champion's weight distribution, the worst genome for contrast, a species breakdown by age and fitness, and the fitness curve so far read back from the CSV.

3.4 Training Approaches

The three approaches below share everything described in Section 3.3 and differ only in what a genome is evaluated on and how its fitness is aggregated.

3.4.1 Approach I: Level-Specific NEAT Training

Our first experiment trains an independent population for each level:

$$\begin{aligned}
&\text{Level 1-1} \rightarrow \text{Population} \rightarrow \text{Evolution} \rightarrow \text{Best Level 1-1 genome}\\
&\text{Level 2-1} \rightarrow \text{Population} \rightarrow \text{Evolution} \rightarrow \text{Best Level 2-1 genome}\\
&\text{Level 3-1} \rightarrow \text{Population} \rightarrow \text{Evolution} \rightarrow \text{Best Level 3-1 genome}
\end{aligned}$$

Each population sees exactly one level for its entire run, and we ran this separately for Levels 1-1, 2-1 and 3-1. The question we wanted to answer was whether NEAT can evolve a specialised agent that maximises performance on an individual level, and it can. This gives us a specialist baseline: an upper bound on what is achievable when generalisation is not required at all.

3.4.1.1 Fitness Function

The scoring here is a speedrun fitness. It rewards distance and velocity, rewards level completion heavily with a time bonus on top, and penalises death:

$$F_{\text{level}} = \max(0.001,\ X_{\max} + 5\,V + B_c - 50\,N_d) \tag{3.1}$$

where $X_{\max}$ is the furthest point Mario reached, N_d the number of deaths, and V his average speed in distance per second,

$$V = \frac{X_{\max}}{f/60} \tag{3.2}$$

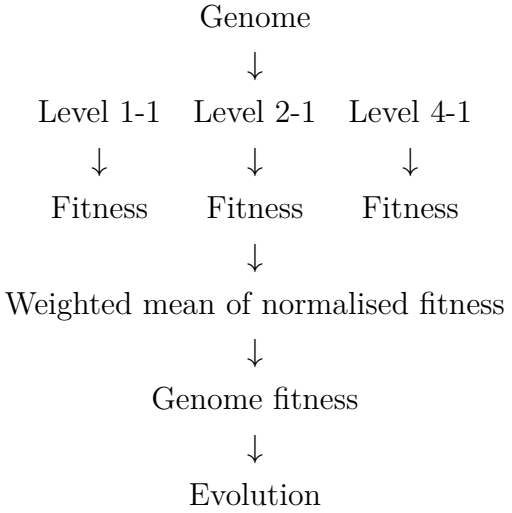
with f the frames elapsed and 60 the frame rate. The completion bonus is paid only when the flagpole is reached,

$$B_c = \begin{cases} 2000 + 2\,(F_{\max} - f), & \text{level completed} \\ 0, & \text{otherwise} \end{cases} \tag{3.3}$$

where $F_{\max}$ is the episode frame cap. The death penalty applies only when the level was not completed, so the two branches are mutually exclusive. The whole expression is floored at 0.001 because NEAT's reproduction requires positive fitness values.
The velocity coefficient of 5 is what makes this a speedrun objective rather than a distance objective. At that weight a genome finishing the same stretch faster gains meaningfully, which is the core signal we wanted, and the completion bonus of 2000 sits far above any distance a genome could accumulate without finishing, so reaching the flagpole always dominates.
The advantage is depth. A population that only ever sees one layout can commit entirely to it and learns that specific sequence of obstacles thoroughly. The disadvantage is that this is close to memorisation: the resulting genome performs poorly on any level it was not trained on, because nothing in its training rewarded behaviour that transfers, and we end up with a separate genome per level rather than one agent that plays the game. This is the same limitation reported for MarI/O [10], and reproducing it deliberately gave us a reference point for judging the other two approaches.

3.4.2 Approach II: Multi-Level Generalised NEAT Training

The second approach changes what a genome must do before it is scored. Rather than one genome per level, a single genome plays every level in the training set and receives one combined fitness:



The methodological point is that the same genome must perform across several levels before any fitness is assigned to it. Selection therefore favours genomes that are consistently competent rather than genomes that are exceptional on one layout and helpless elsewhere, which is exactly the failure mode of Approach I. This approach exists to investigate generalisation directly. Because each level is an independent episode, the evaluations are run in parallel across cores.

3.4.2.1 State Variables

The guards in this fitness function are defined over quantities tracked frame by frame during an episode, so we state them first. Mario's horizontal position at frame t is assembled from two RAM bytes,

$$X_t = 256\,P_t + S_t\,, \qquad X_{\max} = \max_{0\le t\le T} X_t \tag{3.4}$$

where P_t is the page byte and S_t the sub-position byte. The stagnation counter records consecutive frames without new forward progress,

$$s_t = \begin{cases} 0, & X_t > X_{\max,\,t-1} \\ s_{t-1}+1, & X_t \le X_{\max,\,t-1} \end{cases} \tag{3.5}$$

and an episode is abandoned once s_t exceeds 300 frames. Horizontal speed is stored as a signed byte and recovered as

$$v_t = \begin{cases} V_t, & V_t \le 127 \\ V_t - 256, & V_t > 127 \end{cases} \tag{3.6}$$

which gives the speed-violation flag

$$V_{\text{viol}} = \mathbf{1}[\,\exists\, t:\ V_t = \texttt{0xD8}\,] \tag{3.7}$$

Once set, this remains true for the rest of the episode and the run is scored as a death. Deaths are the drop in the lives counter, $N_d = \max(0,\, L_0 - L_T)$, and completion is $C = \mathbf{1}[\, F_T = 3\,]$, where F_T is the float-state byte that reads 3 on the flagpole.

3.4.2.2 Fitness Function

Raw pixel distance is not comparable between levels of different lengths, so distance is expressed as a fraction of the level's own length,

$$P = \frac{X_{\text{credited}}}{L} \tag{3.8}$$

where L is the level length and X_{credited} is the distance travelled excluding any ground covered while a scripted recovery maneuver was driving. The fitness for one level is then

$$F_{\text{norm}} = \max(0.001,\; 1000\,P + V_b + J_b + B'_c - D_p) \tag{3.9}$$

with the velocity bonus gated and capped,

$$V_b = \begin{cases} \min\left(500,\; 500\,\dfrac{P}{f/60}\right), & P \geq 0.05 \\ 0, & \text{otherwise} \end{cases} \tag{3.10}$$

a small bounded reward for jumping while stuck, $J_b = \min(4,\; 0.02\,j)$ where j counts airborne frames while pinned, a completion bonus scaled by the share of distance the genome covered unaided,

$$B'_c = C \cdot E \left(300 + 200\,\frac{F_{\max} - f}{F_{\max}}\right), \qquad E = 1 - \frac{X_{\text{assisted}}}{X_{\max}} \tag{3.11}$$

and a proportional death penalty D_p removing half the accumulated fitness rather than a flat amount.

Almost every term here is bounded, and that is deliberate. Because these per-level scores are averaged, a single unbounded term on one level would let that level dominate the mean. Three of the bounds came from watching specific failures. The velocity gate exists because progress per second grows without limit as an episode shortens, so before it a genome dying at frame 30 scored as much velocity as one that ran a third of the level, and the fastest way to look fast was to barely move and stop. The jump reward is capped below the value of one tile of real progress so it can never outbid advancing, but it exists at all because two genomes pinned at the same pipe were otherwise indistinguishable and selection had nothing to work with. The death penalty is proportional because a flat one collapsed every early genome onto the floor value and erased the gradient in generation 0 entirely.

3.4.2.3 Level Weighting

We weight the levels rather than averaging them flat:

$$F_{\text{genome}} = \frac{\sum_l w_l F_{\text{norm},l}}{\sum_l w_l} \tag{3.12}$$

Once a level is reliably solved it stops carrying information about which genome is better, and giving it equal say lets it dominate a score that should be driven by the levels still unsolved. In our runs Level 4-1 was solved well before the others, so we set $w = 0.4$ for it while leaving Levels 1-1 and 2-1 at $w = 1.0$, with Level 2-1 being the real bottleneck.

3.4.3 Approach III: Moment-Based NEAT Training

The third approach changes the starting condition of evaluation itself, and it is the one we took furthest. It follows the training scheme proposed by Autin [11].
The problem it addresses is structural. When every evaluation starts at the opening frame of a level, the population practises the first stretch on every episode of every generation, while the difficult section halfway through is reached only by the few genomes already good enough to get there. Evolutionary pressure is spent overwhelmingly on ground the population has already mastered.
To address this we first collected a library of mid-level situations, which we refer to as moments, and evaluated genomes starting from those rather than from a single fixed opening state:

Scout gameplay
↓
State collection
↓
Moment dataset (Moment 1 … Moment N)
↓
NEAT population
↓
Each genome evaluated on every moment
↓
Aggregate fitness
↓
Selection and next generation

3.4.3.1 Collecting the Moments

A moment is a snapshot of emulator state taken partway through a level, which can be reloaded to drop Mario directly into that situation: halfway through the pipe sequence in 2-1, mid-descent onto an enemy, partway up the stairs in 3-1. Autin [11] built this dataset by playing the game by hand and saving states at interesting points. That part of the method is not reproducible, since it is hours of one person's time at a keyboard and a second attempt would produce a different dataset, so we replaced the human with a scripted scout.

The scout is not trained and is never scored. Its only job is to walk each level so the collector can snapshot the emulator every 150 pixels of forward progress. We gave it the same 16×13 tile grid the networks see rather than letting it react to raw position, because a scout that simply holds RIGHT and jumps when progress stalls is reacting after the fact and dies at the first enemy. Reading the grid lets it look ahead instead, and it means the moments are sampled from states a trained genome could plausibly encounter.

We deliberately kept the scout mediocre. A scout that played each level optimally would only ever sample the optimal path, and the dataset would inherit that bias; some clumsiness means it spends time in awkward positions, which is where the interesting moments are.

Collection runs in waves, because the scout dies well before the flagpole on the harder levels. The first wave plays from the level's start, and each later wave restarts from the furthest moment already banked and continues from there, pushing the frontier deeper. The scout never clears the 2-1 pipes, but it does not need to: reloading a state past them and playing on is enough to sample what lies beyond. Our dataset holds 57 moments across Levels 1-1, 2-1, 3-1 and 4-1.

3.4.3.2 Per-Moment Fitness

Each moment is scored with the function given by Autin [11]. For moment i,

$$F_i = (M_{\text{end},i} - M_{\text{start},i}) - 0.25\, t_i + S_i + B_{\text{goal},i} - P_{\text{fail},i} \tag{3.13}$$

where $M_{\text{end},i} - M_{\text{start},i}$ is the distance travelled from where the moment dropped Mario in, t_i the frames used, S_i a speed bonus, $B_{\text{goal},i} = 1000$ if the flagpole is reached and 0 otherwise, and $P_{\text{fail},i} = 50$ charged when the moment ends without covering its span. Several details here matter. Distance is measured from the moment's own starting position rather than from the level's origin, so a moment beginning deep in a level does not hand out unearned progress. It is capped at a span of 250 pixels, which is where we treat the moment as answered; without that bound a genome that clears its obstacle keeps running into the next moment's territory and is scored on ground

another moment already covers. The frame penalty of $t/4$ rather than $t/2$ prioritises level progress over the speed at which it was made. The speed bonus counts every unit of progress made above walking pace twice. The failure penalty is small by design, since its job is only to separate "did not get there" from "got there slowly", not to overwhelm the progress term.

3.4.3.3 Aggregation I: Mean Across Moments

The first scheme is the one Autin [11] settled on, in which a genome's fitness is the mean of its per-moment scores across all N moments in the batch:

$$F_{\text{mean}} = \frac{1}{N} \sum_{i=1}^{N} \left[(M_{\text{end},i} - M_{\text{start},i}) - 0.25\, t_i + S_i + B_{\text{goal},i} - P_{\text{fail},i} \right] \tag{3.14}$$

Every genome is evaluated on every moment, and this matters more than it appears to. If genomes instead played moments until they died, a hard moment landing early in the shuffled order would end the run and the genome would score near zero regardless of its actual competence. Genomes poorly suited to one situation but strong elsewhere would then be eliminated by ordering luck, and the population would lose that knowledge. Averaging over the full batch removes the luck and asks a stable question: how well does this genome handle a typical situation.

3.4.3.4 Aggregation II: Spread Penalty

The mean has a weakness. It rewards a high average, but says nothing about consistency, so a genome that is excellent on most moments and hopeless on a few can outscore one that is uniformly competent. The second scheme is our own addition rather than part of the scheme in [11], and it penalises that inconsistency:

$$F_{\text{spread}} = \mu_F - \lambda\, \sigma_F \tag{3.15}$$

where

$$\mu_F = \frac{1}{N} \sum_{i=1}^{N} F_i\,, \qquad \sigma_F = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (F_i - \mu_F)^2} \tag{3.16}$$

and F_i is the full per-moment score from Equation 3.13. The difference is easiest to see with an example. A genome scoring $900, 900, 900, 100$ has a mean of 700, while one scoring $650, 650, 650, 650$ has a mean of 650. The mean scheme prefers the first; the spread scheme can prefer the second, because the first genome's variance is large and the penalty charges for it. In practice this makes the trade of selling performance

on one level to gain slightly more on another unprofitable, since that trade always widens the spread.

We also apply this per level rather than per moment where the levels are unevenly represented. Our dataset has 21 moments for Level 1-1 but only 7 for Level 2-1, so a flat mean over moments lets a level contribute in proportion to how many moments it happens to have. Averaging within each level first, then combining the per-level scores, removes that imbalance.

This variant has a known weakness we should state plainly. A uniformly poor genome has a standard deviation near zero and so escapes the penalty entirely. Early in a run, when almost every genome scores near zero, the term is close to constant across the population and contributes little selection pressure, so evolution proceeds effectively on the mean until real differences appear. We kept both aggregation schemes and ran them separately rather than replacing one with the other, so the two can be compared directly.

CHAPTER 4

SYSTEM DESIGN

4.1 Requirement Specification

4.1.1 Functional Requirements

-
-
-

4.1.2 Non-Functional Requirements

-
-
-

4.2 Context Diagram

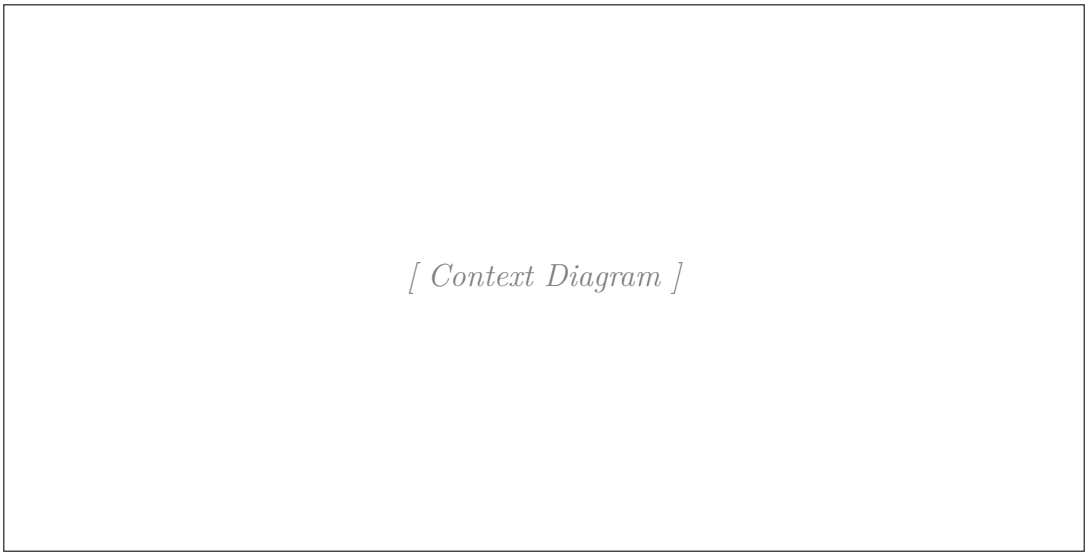


Figure 4.1: Context Diagram

4.3 Data Flow Diagram

4.3.1 DFD Level 0

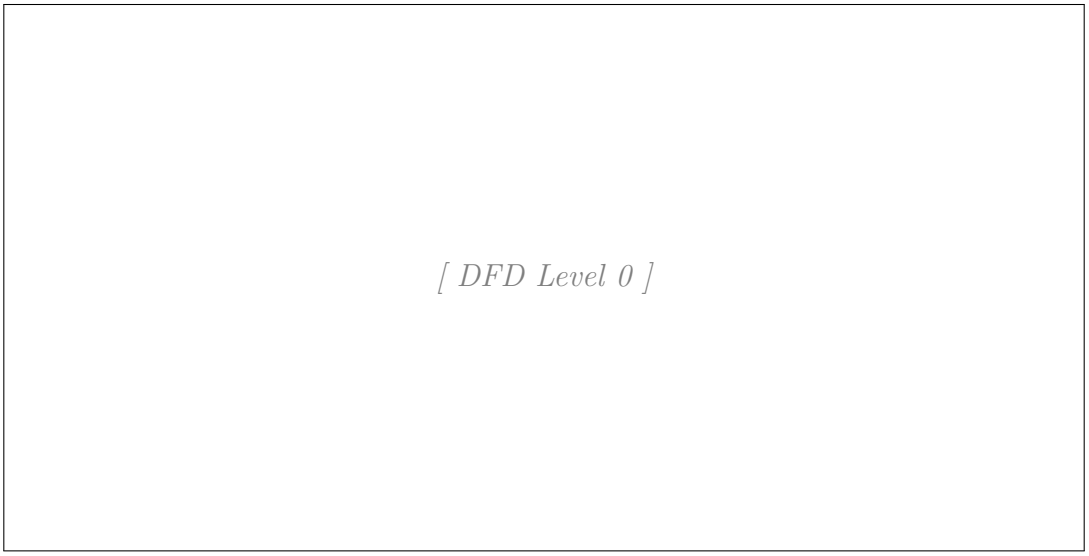


Figure 4.2: DFD Level 0

4.3.2 DFD Level 1

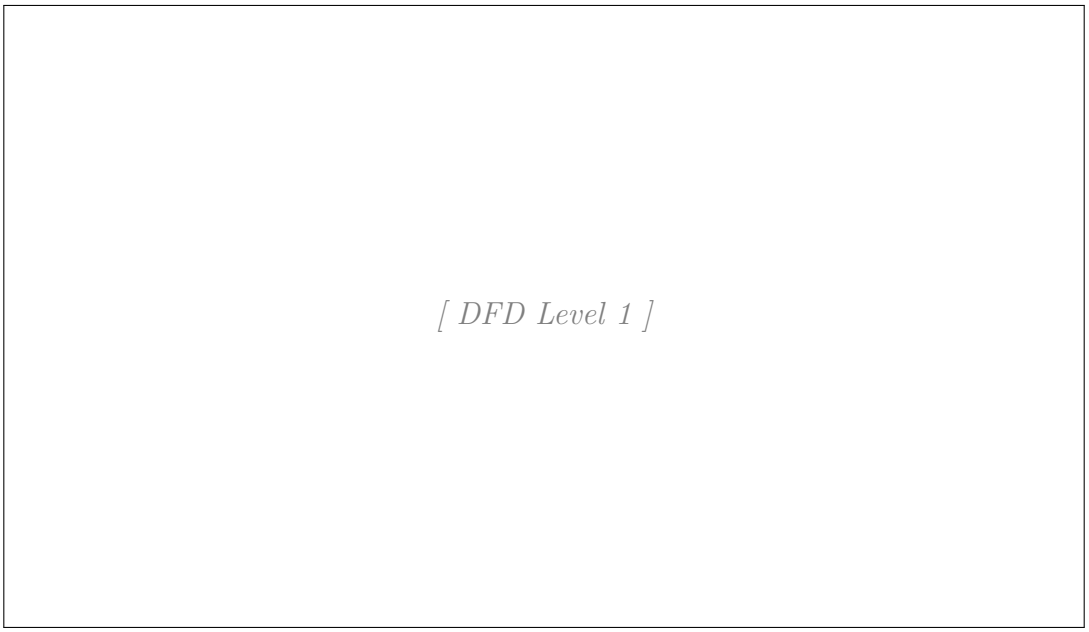


Figure 4.3: DFD Level 1

4.4 Use Case Diagram

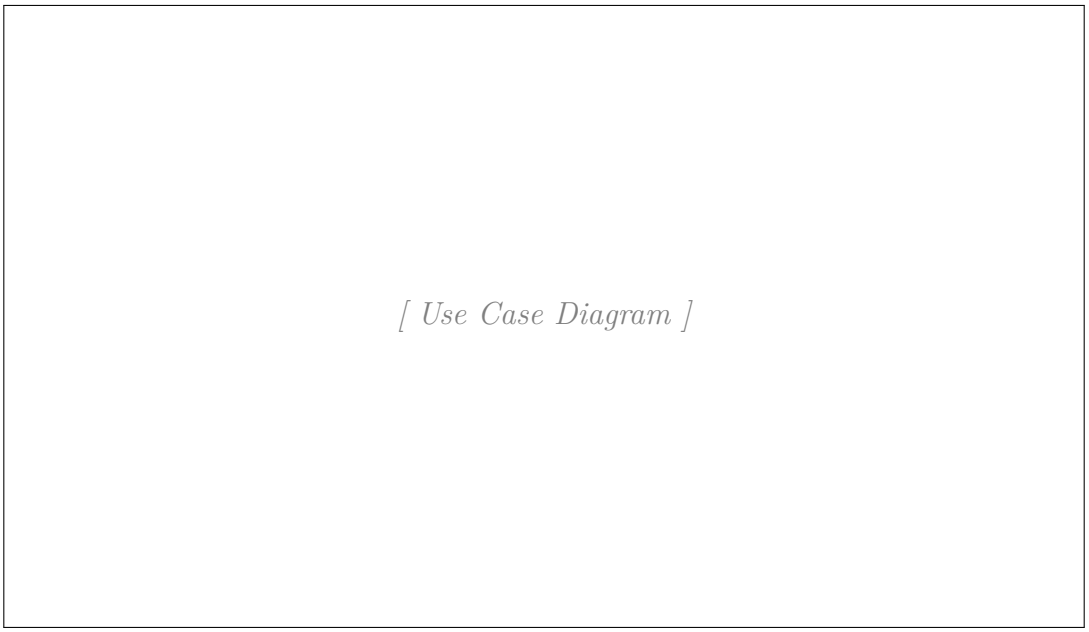


Figure 4.4: Use Case Diagram

4.5 Class Diagram

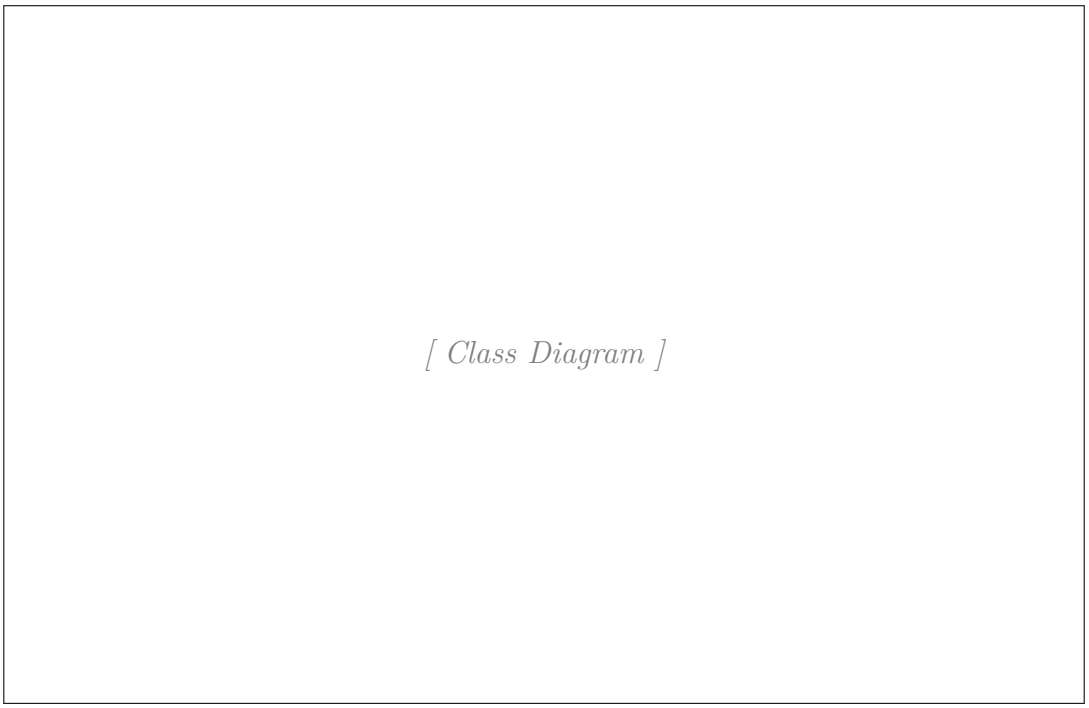


Figure 4.5: Class Diagram

4.6 Sequence Diagram

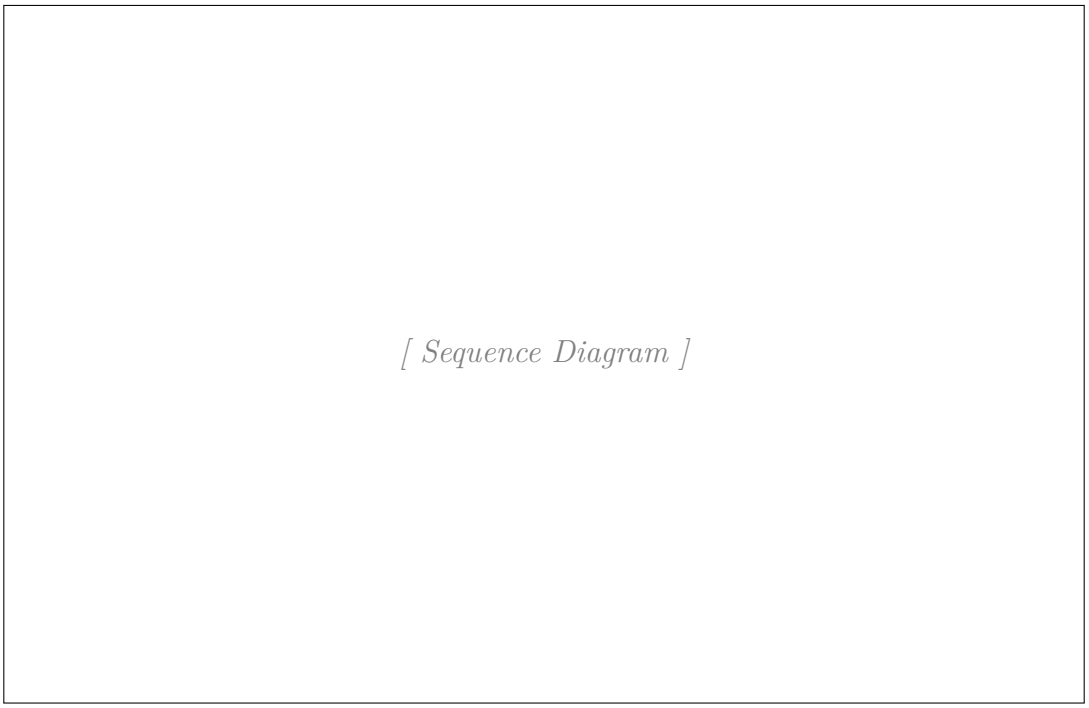


Figure 4.6: Sequence Diagram

4.7 Component Diagram

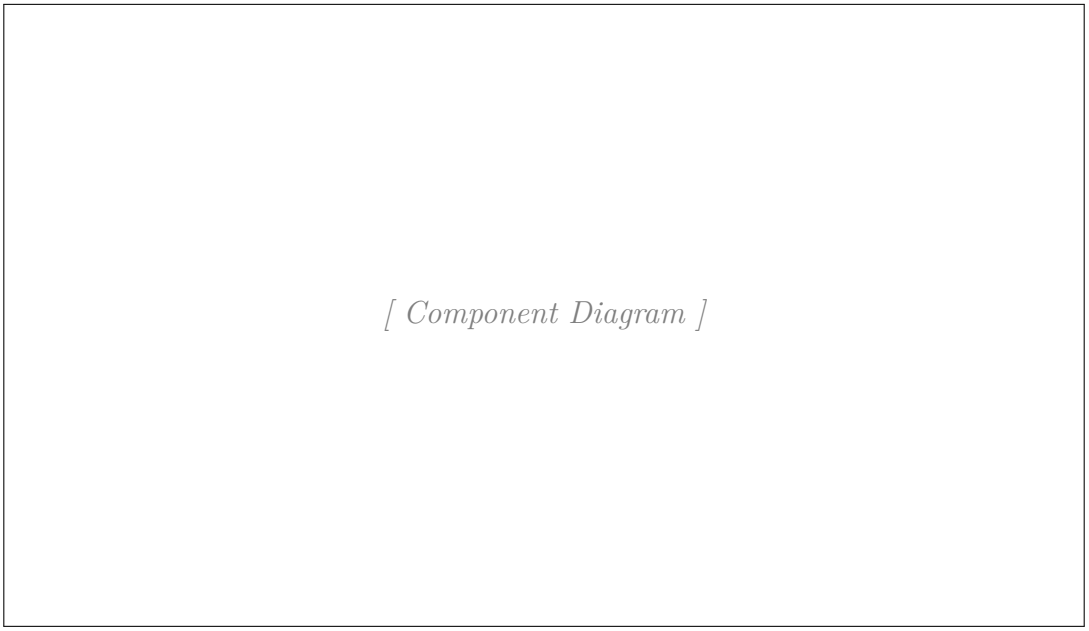


Figure 4.7: Component Diagram

4.8 Deployment Diagram

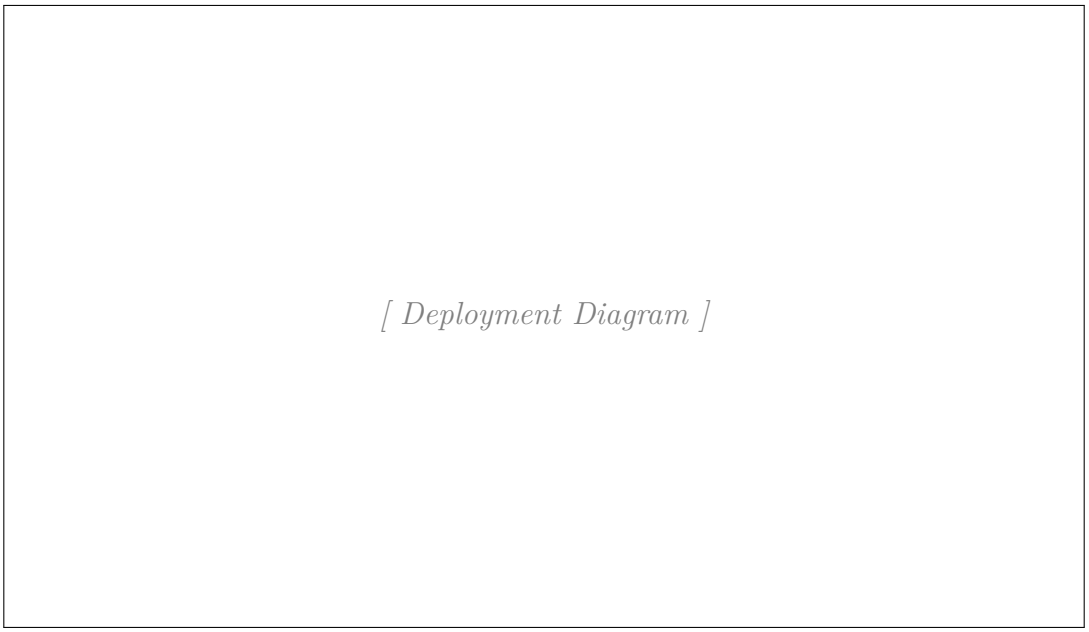


Figure 4.8: Deployment Diagram

4.9 Gantt Chart

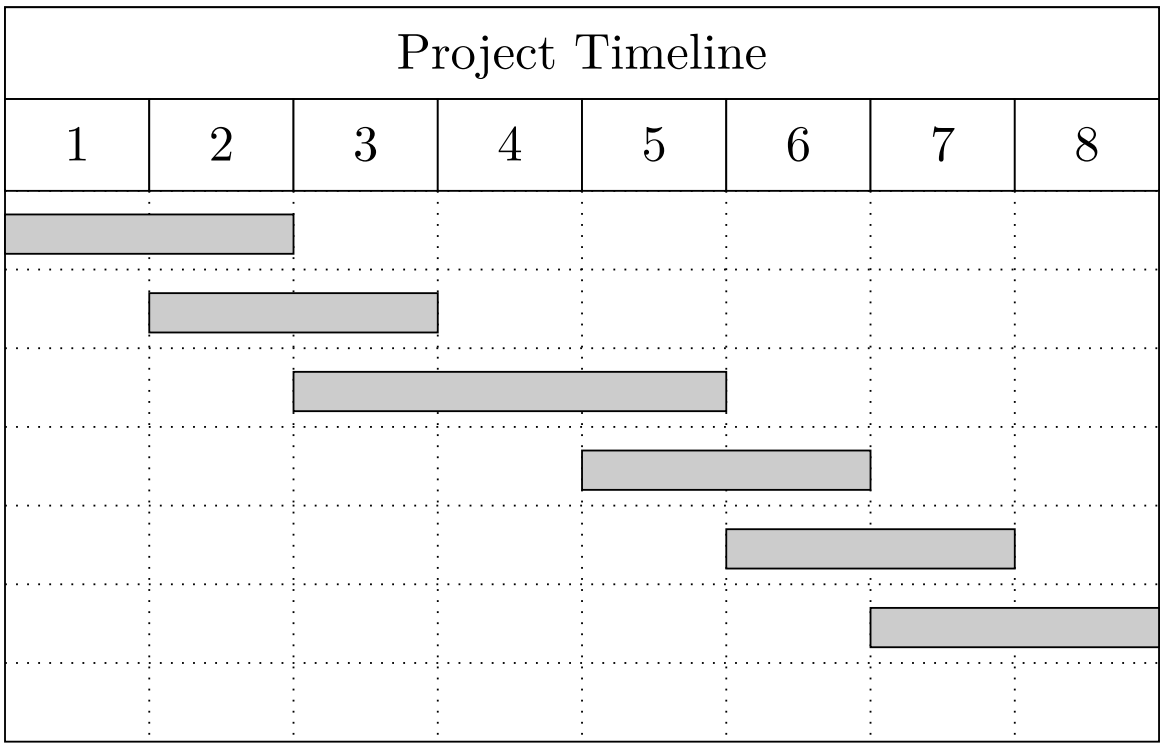


Figure 4.9: Project Gantt Chart

CHAPTER 5

RESULT AND DISCUSSION

5.1 Experimental Setup

All runs were carried out on consumer hardware with training confined to the CPU, using the pipeline described in Section 3.3. Every run logs one row per generation to a CSV file, and the figures in this chapter are drawn from those logs.

Fitness values are not compared across approaches anywhere in this chapter. The three approaches use different fitness functions with different scales and different meanings, so a fitness of 500 under one is not a better or worse result than a fitness of 500 under another. Where a comparison is made across approaches it is made on distance covered, on network structure, or on the consistency of a genome's performance across levels, all of which mean the same thing regardless of how the genome was scored.

Table 5.1 summarises the runs.

Table 5.1: Summary of Training Runs

Approach	Run	Generations	Best distance
I	Level 1-1	1102	3267
I	Level 2-1	744	2458
I	Level 3-1	4054	2962
II	Multi-level	154	3267
III	Mean aggregation	161	—
III	Spread penalty	87	—

Distance is not reported for Approach III because a moment run does not measure it the same way: a genome starts partway through a level and is scored on the ground it covers from there, so the figure is not comparable with a full-level run.

5.2 Approach I: Level-Specific Training

5.2.1 Distance and Level Coverage

Each level was trained with its own population. Figure 5.1 shows the furthest point reached over the course of each run, and Figure 5.2 expresses the same data as a percentage of the level's length so the three runs can be read against one another.

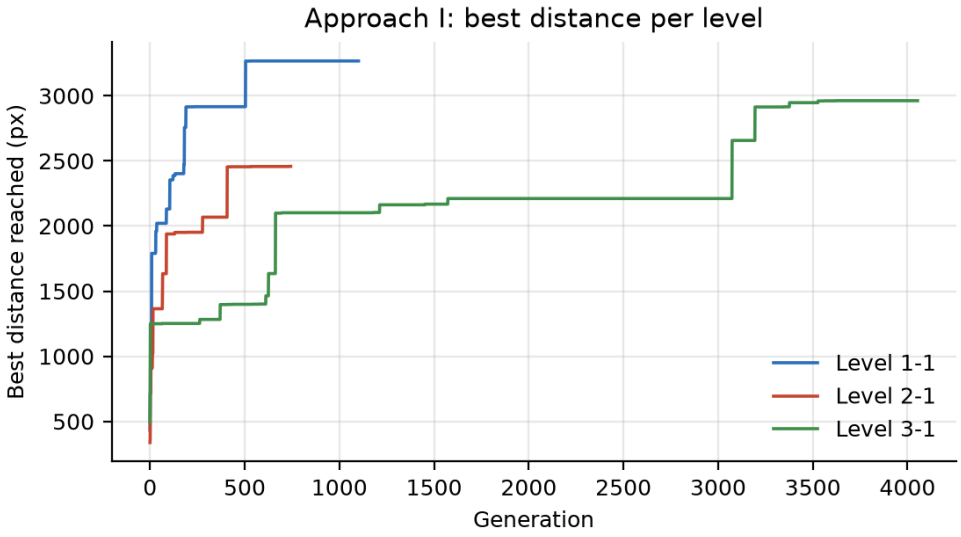


Figure 5.1: Best distance reached per generation, Approach I

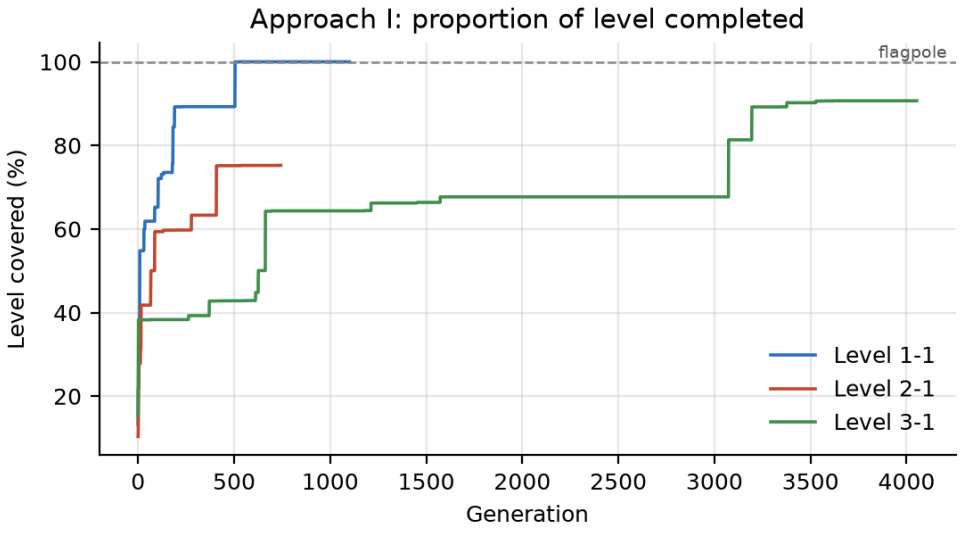


Figure 5.2: Proportion of each level covered, Approach I

Level 1-1 was solved. The population reached the flagpole at generation 500 and held it for the remaining 600 generations of the run. Level 3-1 reached 90.7% of the level, but took 4054 generations to do so and spent most of that time on a plateau: it sat

at roughly 74% from generation 1500 to generation 3000 before a late improvement carried it most of the way. Level 2-1 was the hardest, stalling at 75.3% and not improving over the last 200 generations of its run.

The shape of all three curves is the same and is worth noting: progress is not gradual. Long flat stretches are broken by sudden jumps, which is what evolutionary search looks like when a structural mutation finally produces behaviour that clears an obstacle the population has been stuck against. The flat sections are not wasted time; they are the population accumulating variation that has not yet paid off.

5.2.2 Structural Complexity

Figure 5.3 shows how the champion network's structure developed.

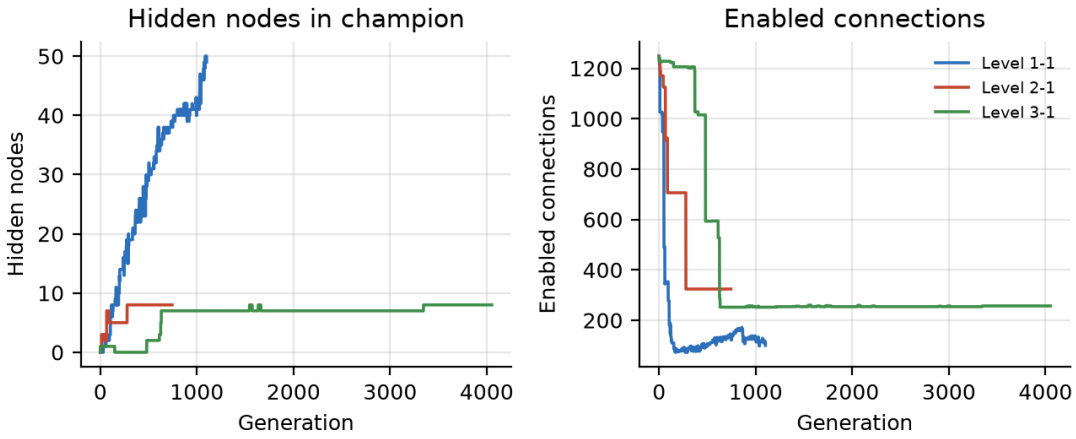


Figure 5.3: Champion network structure over training, Approach I

This is NEAT complexifying as intended. Every run starts from the same place, zero hidden nodes and 1248 enabled connections, since the initial genomes are fully connected with no hidden layer. From there the two quantities move in opposite directions. Hidden nodes accumulate, reaching 50 on Level 1-1 and 8 on the other two runs. Enabled connections fall sharply, from 1248 down to around 100 on Level 1-1 and 250 to 320 on the others.

The network is therefore not growing in the naive sense. It is discarding most of the dense input-to-output wiring it started with and replacing a small part of it with hidden structure. What survives is a much sparser network with a few intermediate nodes, which is a more useful description of the policy than the fully connected sheet it began as. Level 1-1, the run that actually solved its level, is also the run that went furthest in both directions, adding the most hidden nodes and pruning to the fewest connections.

5.2.3 Learning Rate

Figure 5.4 normalises each run against its own starting distance and plots it against the fraction of the run completed, which makes the three learning curves comparable despite their very different lengths.

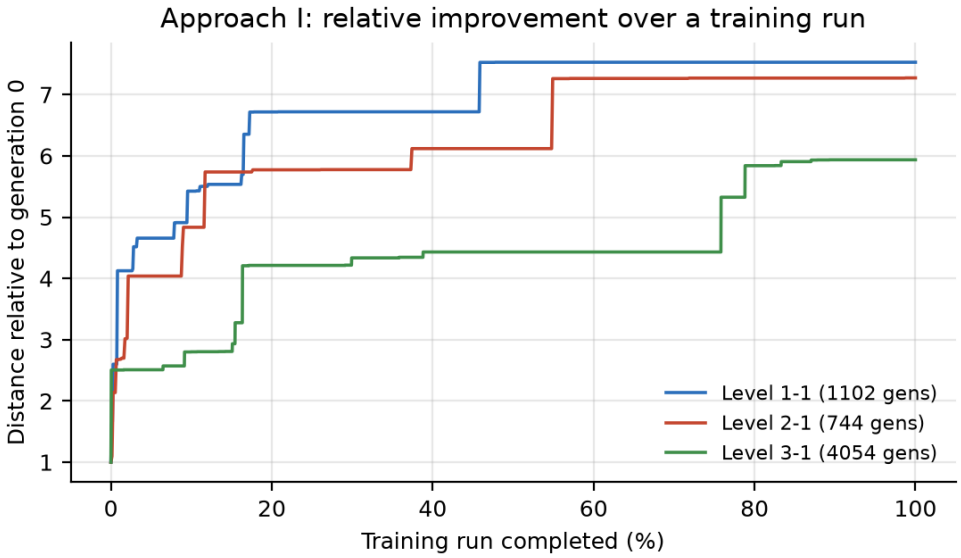


Figure 5.4: Relative improvement over a training run, Approach I

All three runs gain most of their improvement early. By the time 20% of each run has elapsed, Level 1-1 has already improved 6.7-fold over its generation-0 distance and Level 2-1 5.8-fold, and the remaining 80% of the run adds comparatively little. This is the practical argument against simply training for longer: Level 3-1 ran for 4054 generations, nearly four times Level 1-1's run, and finished with a smaller relative gain.

5.2.4 Limitation

The limitation of this approach is the one the curves cannot show. Each of these genomes was evolved against a single fixed layout and is scored only on that layout, so nothing in training rewarded behaviour that transfers. The Level 1-1 champion has learned the specific sequence of obstacles in Level 1-1, not how to play Super Mario Bros., and it performs poorly when placed in any other level. This is the same overfitting reported for MarI/O [10], and it is what motivated the second approach.

5.3 Approach II: Multi-Level Training

Approach II evaluates one genome across several levels and combines the results into a single score, so that selection favours genomes that are competent everywhere rather than genomes tuned to one layout. The run analysed here resumes from generation 806 of an earlier run and continues to generation 959.

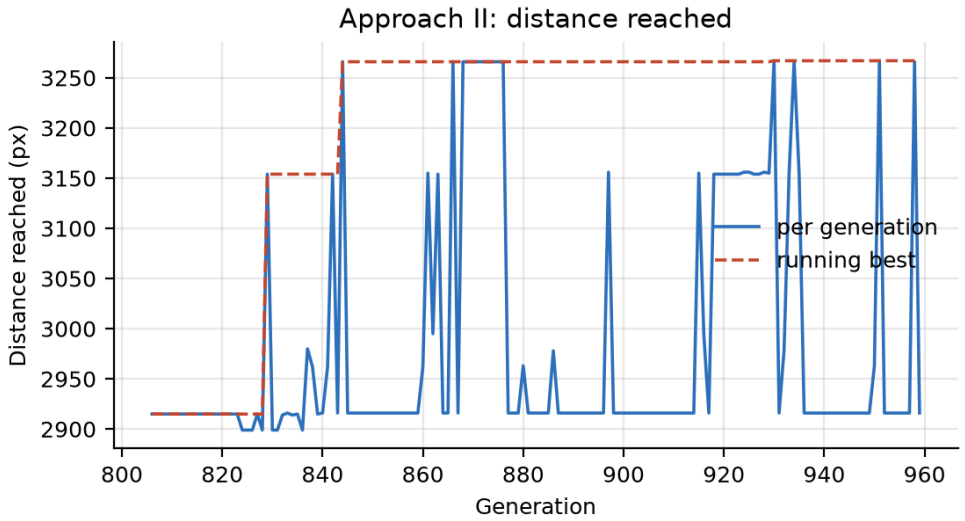


Figure 5.5: Distance reached per generation, Approach II

Figure 5.5 shows distance holding between roughly 2900 and 3267 across the window, with the running best reaching the full level length. The per-generation trace varies considerably from one generation to the next while the running best barely moves, which is the signature of a population that has converged: the champion is stable and the variation is the rest of the population exploring around it.

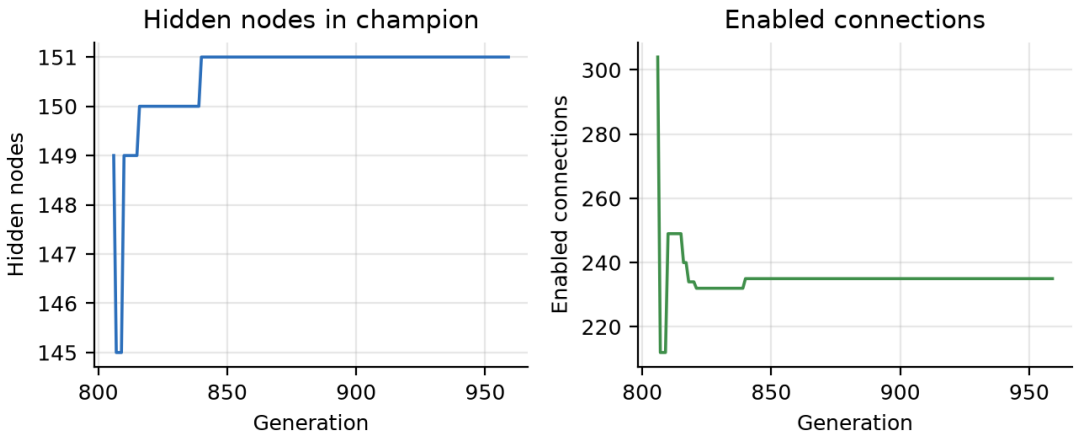


Figure 5.6: Champion network structure, Approach II

The structural picture in Figure 5.6 is markedly different from Approach I. The champion carries around 150 hidden nodes, far more than the 8 to 50 seen in the level-specific runs, while its enabled connections continue to fall, from 304 to 235 over the window. A genome that has to handle several levels ends up with substantially more internal structure than one tuned to a single layout, which is what one would expect if the extra nodes are encoding responses to situations that only arise on some of the levels.

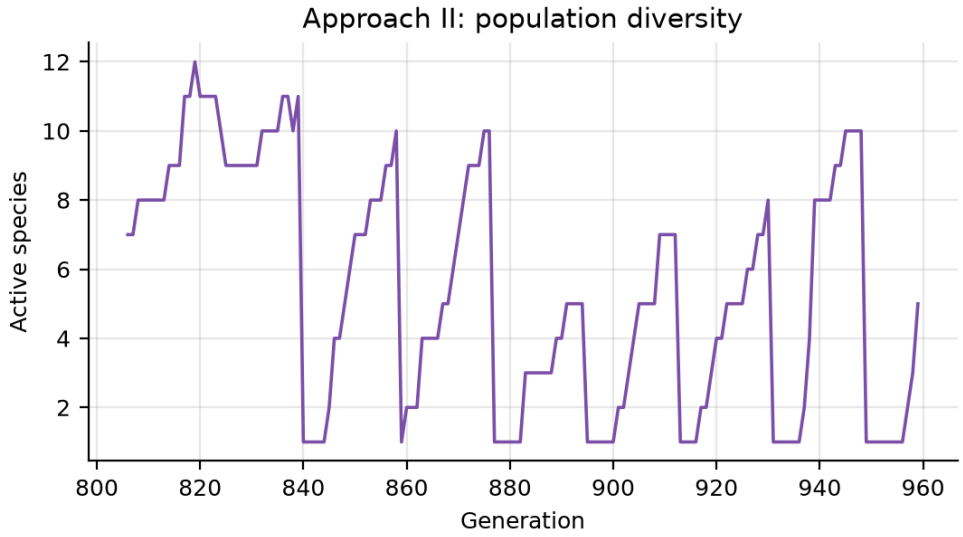


Figure 5.7: Active species count, Approach II

Figure 5.7 shows the species count moving between 1 and 12. The periods where it falls to a very low value are the population converging onto a single strategy, and the recoveries afterwards are the reset mechanism described in Section 3.4.2 rebuilding diversity. The mechanism is visibly doing its job here, though the fact that it has to intervene repeatedly indicates how strongly this configuration tends toward convergence.

5.4 Approach III: Moment-Based Training

5.4.1 Mean Aggregation

The first moment-based scheme scores a genome as the plain mean of its per-moment fitness across the whole batch. Figure 5.8 plots the champion's fitness on each of the four levels separately, which is what makes the problem with this scheme visible.

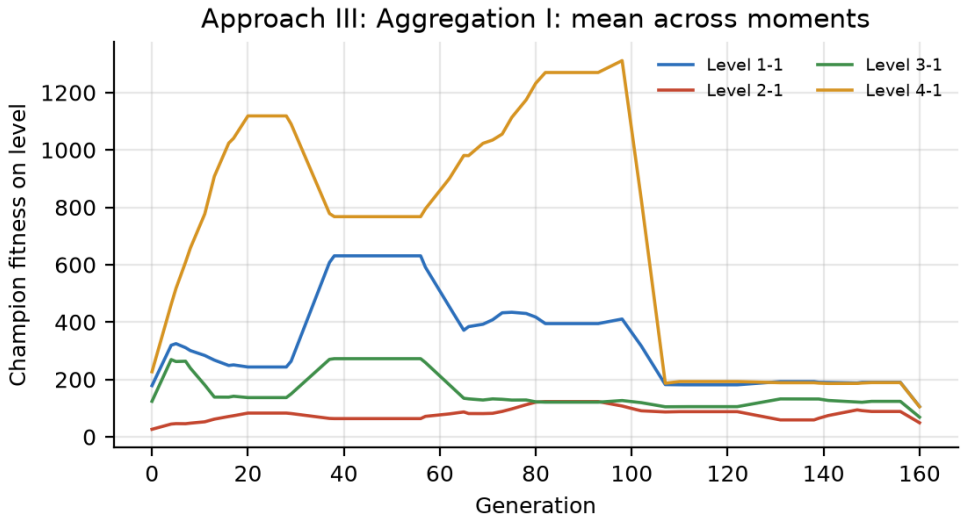


Figure 5.8: Per-level champion fitness under mean aggregation

The four curves separate and stay separated. Averaged over the last ten generations of the run, the champion scores 190.2 on Level 1-1 and 189.2 on Level 4-1, but only 88.5 on Level 2-1, a spread of 101.7 between its best and worst level.

This is exactly the failure the mean invites. A genome is selected on its average, so a large gain on one level fully compensates a large loss on another. Selection is therefore indifferent between a genome that plays all four levels adequately and one that plays two of them well and a third badly, provided the averages match. Since some levels are easier to improve on than others, the population drifts toward the easy ones and lets the hard one degrade, which is what the gap between the Level 1-1 and Level 2-1 curves records.

5.4.2 Spread Penalty

The second scheme subtracts a multiple of the standard deviation across levels from the mean, so a genome is rewarded for being good and for being evenly good.

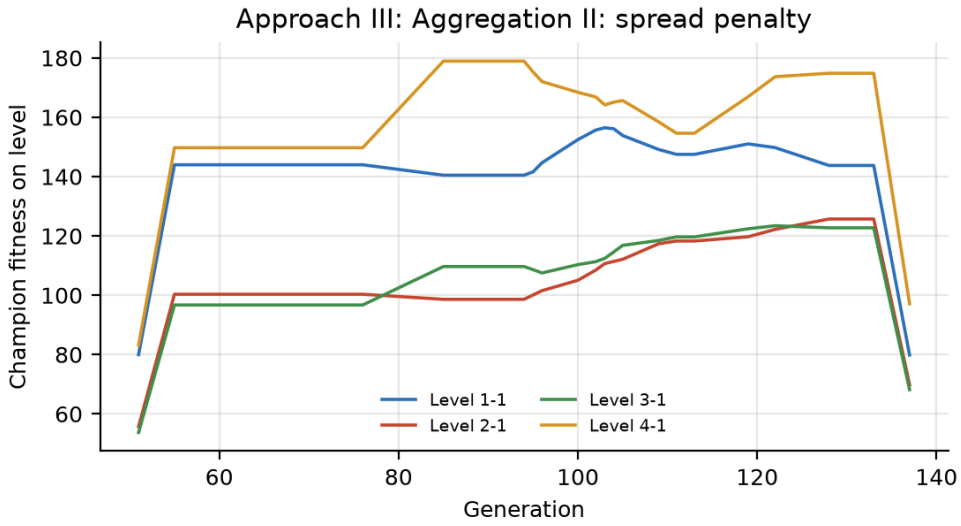


Figure 5.9: Per-level champion fitness under the spread penalty

Figure 5.9 shows the four curves running much closer together. Over the last ten generations the champion scores 143.8, 125.7, 122.7 and 174.8 on Levels 1-1 through 4-1, a spread of 52.1 against the 101.7 seen under the mean. The important part is where the change came from: Level 2-1, the weakest level under the mean, improves from 88.5 to 125.7, while Level 1-1 falls from 190.2 to 143.8. The penalty has traded peak performance on the easy levels for competence on the hard one, which is precisely what it was designed to do.

5.4.3 Consistency

Figure 5.10 states the comparison directly, plotting the standard deviation of the champion's fitness across the four levels for both schemes. Both curves are restricted

to generations 51 to 137, the window the two runs share, so they are compared over identical ground.

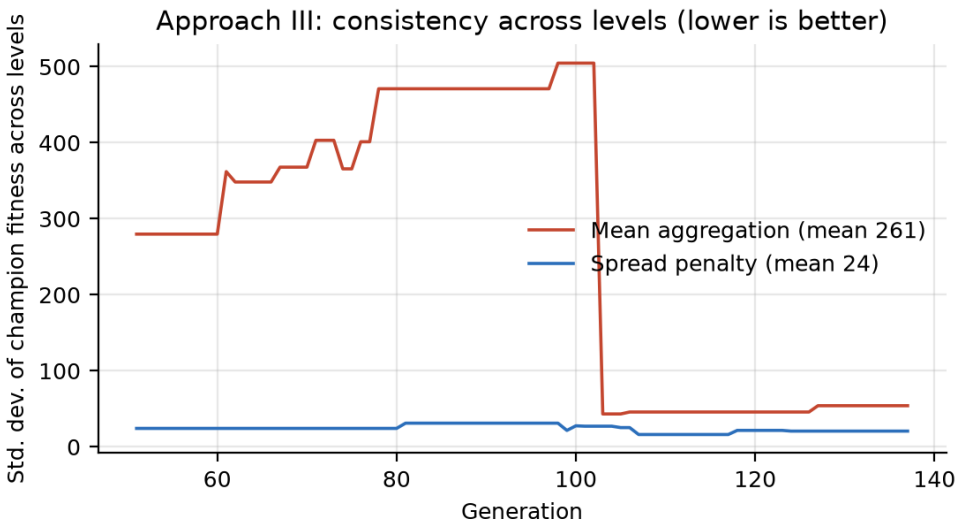


Figure 5.10: Consistency across levels under both aggregation schemes

Over that window the mean scheme averages a cross-level standard deviation of 261, against 24 for the spread penalty, a reduction of roughly eleven times. The mean scheme's curve is also erratic, climbing above 500 around generation 100 before dropping sharply, which reflects the champion changing to a genome with a quite different balance across levels. The spread penalty's curve stays flat and low for its entire run. The mean distance covered per moment supports the same conclusion. Under the mean scheme it falls over the run, from 230 to 188, while under the spread penalty it rises from 163 to 177. A scheme that was rewarding genuine improvement should not see its distance measure decline, and the fact that the mean scheme does is a further sign that it was optimising the score rather than the behaviour.

One qualification should be stated. The spread penalty run is shorter, 87 generations against 161, and it resumes from an existing checkpoint rather than starting fresh, so the comparison is between the two schemes over a shared window rather than between two complete runs from scratch. The size of the difference, an order of magnitude in cross-level deviation, is large enough that this does not change the conclusion, but a longer paired run would state it more firmly.

5.5 Discussion

The three approaches form a sequence in which each one addresses the limitation the previous one exposed.

Approach I established that NEAT can evolve a competent agent for a fixed level, solving Level 1-1 outright and covering most of Levels 2-1 and 3-1. It also established the cost of doing so: those genomes are specialists, they do not transfer, and one has to be trained per level. The result is a good baseline and a poor agent.

Approach II removed the per-level specialisation by requiring one genome to play several levels before it is scored. This produced champions with substantially more internal structure, consistent with a policy that has to cover more situations, and the diversity trace shows the population repeatedly converging and having to be rebuilt, which suggests the multi-level objective is harder to satisfy than the single-level one.

Approach III changed what a genome practises rather than what it is scored on, evaluating from mid-level situations so that difficult sections are trained as often as the opening stretch. Doing this exposed a problem that Approach II's combined score had hidden: with a plain mean, consistency across levels is not actually being selected for, and the population will quietly trade a hard level away. The spread penalty corrects this and reduces cross-level deviation by roughly eleven times, at a deliberate cost to peak performance on the easier levels.

The broader observation across all three is that the fitness function, not the network, was where nearly every problem in this project was found and fixed. Each of the failures documented here, the specialist that will not transfer, the population that converges, and the mean that trades away a level, was a case of the agent optimising exactly what it was asked to optimise. The corrections were changes to the question, not to the learner.

CHAPTER 6

CONCLUSION AND RECOMMENDATION

6.1 Conclusion

6.2 Recommendations

-
-
-

6.3 Limitations and Future Work

-
-
-

BIBLIOGRAPHY

[1] C. E. Shannon, "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 41, no. 314, pp. 256–275, 1950.

[2] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, 1959.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.

[5] J. Lehman and K. O. Stanley, "Abandoning objectives: An alternative principle for artificial intelligence," *Evolutionary Computation*, vol. 19, no. 2, pp. 189–223, 2011.

[6] S. Karakovskiy and J. Togelius, "The Mario AI benchmark and competitions," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 55–67, 2012.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[8] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017, pp. 16–17.

[9] X. Yao, "Evolving artificial neural networks," *Proceedings of the IEEE*, vol. 87, no. 9, pp. 1423–1447, 1999.

[10] SethBling, "MarI/O – machine learning for video games," YouTube video, 2015. [Online]. Available: `https://www.youtube.com/watch?v=qv6UVOQ0F44`

[11] R. A. Autin, "Super Mario evolution by the augmentation of topology," Master's thesis, University of New Orleans, 2024. [Online]. Available: `https://scholarworks.uno.edu/td/3161/`

[12] J. Togelius, S. Karakovskiy, and R. Baumgarten, "The 2009 Mario AI competition," *IEEE Computational Intelligence Magazine*, vol. 5, no. 2, pp. 16–29, 2010.

[13] M. Yu, "Super Mario Bros reinforcement learning with RAM state extraction," 2020. [Online]. Available: `https://github.com/yumouwei/super-mario-bros-reinforcement-learning`

[14] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, "Manifesto for agile software development," 2001. [Online]. Available: `http://agilemanifesto.org/`

[15] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA: Pearson, 2016.

Appendix

A. User Interface Screenshots

A1.

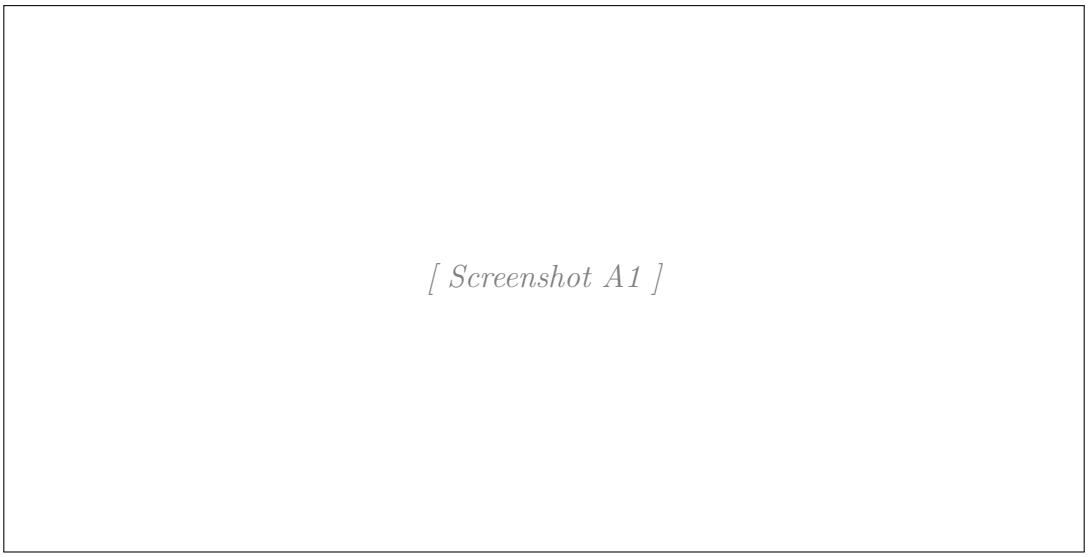


Figure 6.1

A2.

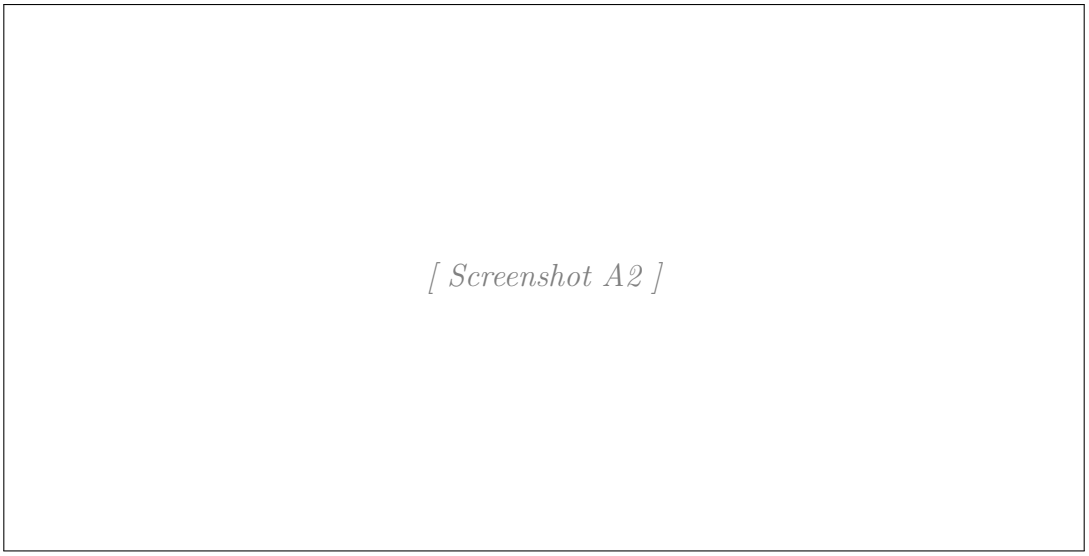


Figure 6.2

A3.

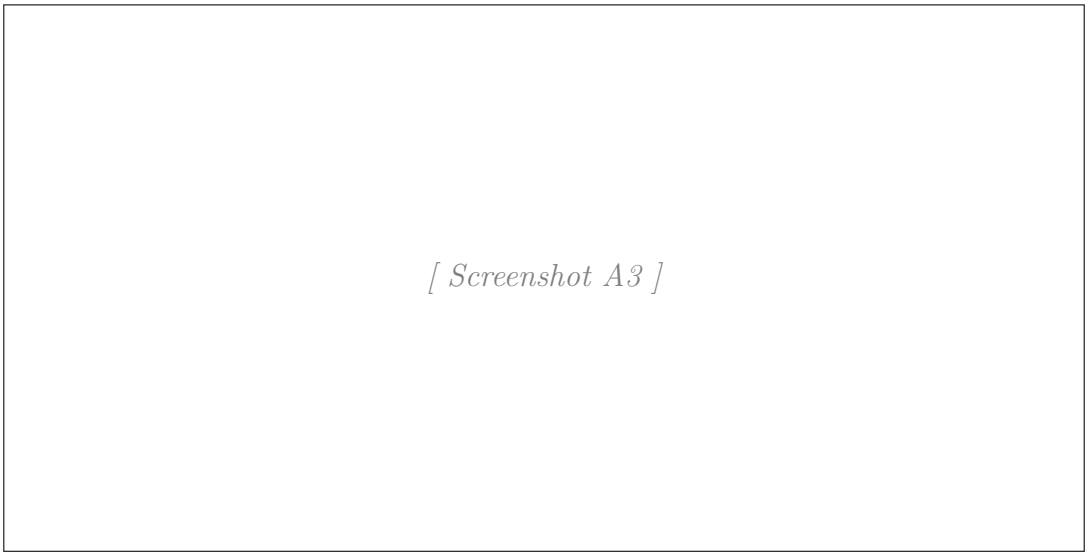


Figure 6.3

A4.

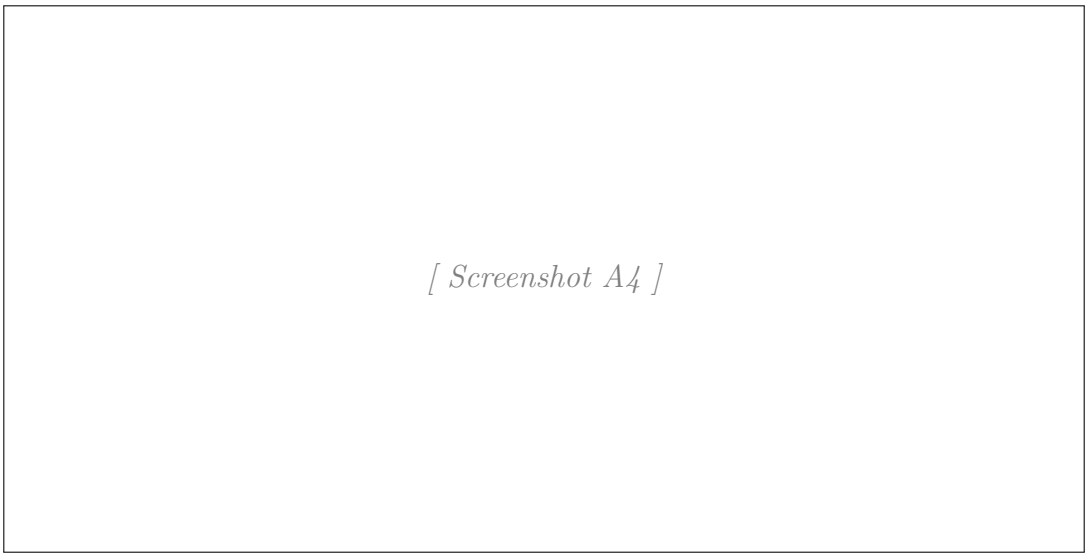


Figure 6.4

A5.

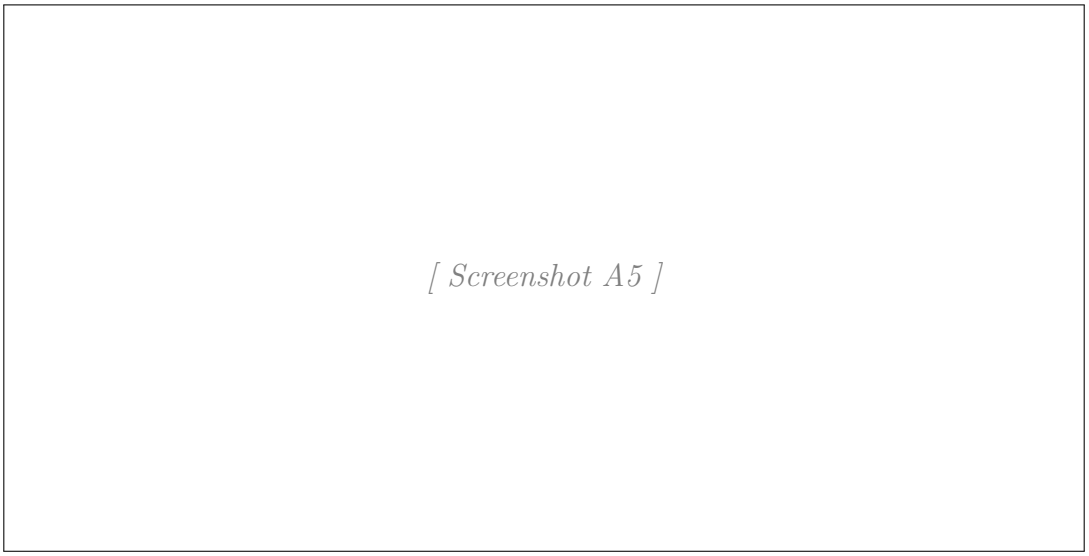


Figure 6.5